~\OneDrive\Desktop\4 year 1 semester\IT4123(P) Agent Based Computing\for_practice\order\BookClientAgent.java

```java
import jade.core.Agent;
import jade.core.AID;
import jade.core.behaviours.*;
import jade.lang.acl.ACLMessage;
import jade.domain.DFService;
import jade.domain.FIPAException;
import jade.domain.FIPAAgentManagement.DFAgentDescription;
import jade.domain.FIPAAgentManagement.ServiceDescription;
import java.util.*;

public class BookClientAgent extends Agent {
    // The title of the book to buy
    private String name;
    private Integer quantity;
    // The list of known seller agents
    private AID serverAgents;

    protected void setup() {
        System.out.println("Hallo! Buyer-agent " + getAID().getName() + " is ready.");

        // Get the title of the book to buy as a start-up argument
        Object[] args = getArguments();
        if (args != null && args.length >= 2) {  // ensure both args exist
            name = args[0].toString();
            quantity = Integer.parseInt(args[1].toString());
            System.out.println("Order name is: " + name + " : " + quantity);
        } else {
            System.err.println("No arguments provided! Agent terminating.");
            doDelete();
            return;
        }

        // Search for seller agents
        DFAgentDescription template = new DFAgentDescription();
        ServiceDescription sd = new ServiceDescription();
        sd.setType("book-selling");
```

```
        template.addServices(sd);

        try {
            DFAgentDescription[] result = DFService.search(this, template);
            if (result.length > 0) {
                serverAgents = result[0].getName();
                System.out.println(getLocalName() + ": Found " + serverAgents.getLocalName());
                addBehaviour(new OrderRequest());
            } else {
                System.err.println(getLocalName() + ": OrderService not found");
                doDelete();
            }
        } catch (FIPAException fe) {
            fe.printStackTrace();
        }
    }

    protected void takeDown() {
        System.out.println("Buyer-agent " + getAID().getName() + " terminating.");
    }

    private class OrderRequest extends Behaviour {
        private boolean finished = false;
        private int step = 0;

        public void action() {
            switch (step) {
                case 0:
                    ACLMessage menuRequest = new ACLMessage(ACLMessage.REQUEST);
                    menuRequest.addReceiver(serverAgents);
                    menuRequest.setContent("MENU");
                    send(menuRequest);
                    step = 1;
                    break;

                case 1:
                    ACLMessage reply = receive();
                    if (reply != null) {
```

```
                displayMenu(reply.getContent());
                step = 2;
            } else {
                block();
            }
            break;

        case 2:
            ACLMessage order = new ACLMessage(ACLMessage.REQUEST);
            System.out.println("\n" + getLocalName() + ": Placing order for " + name + " x" + quantity);
            order.addReceiver(serverAgents);
            order.setContent("ORDER|" + name + "|" + quantity);
            send(order);
            step = 3;
            break;

        case 3:
            ACLMessage orderReply = receive();
            if (orderReply != null) {
                handleRequest(orderReply);
                step = 4;
            } else {
                block();
            }
            break;

        case 4:
            ACLMessage history = new ACLMessage(ACLMessage.REQUEST);
            history.addReceiver(serverAgents);
            history.setContent("HISTORY");
            send(history);
            step = 5;
            break;

        case 5:
            ACLMessage historyReply = receive();
            if (historyReply != null) {
                handlHistory(historyReply.getContent());
```

```
                        finished = true;
                        step = 6;
                    } else {
                        block();
                    }
                    break;
            }
        }

        public boolean done() {
            return finished;
        }
    }

    private void displayMenu(String menuContent) {
        System.out.println("\n" + getLocalName() + ": Received menu");
        String[] items = menuContent.split("\\|\\|");
        for (int i = 1; i < items.length; i++) { // skip "Menu" header
            if (items[i].trim().isEmpty()) continue;
            String[] parts = items[i].split(":");
            if (parts.length < 2) continue; // safety check
            System.out.println(parts[0] + ":" + parts[1]);
        }
    }

    private void handleRequest(ACLMessage reply) {
        String content = reply.getContent();
        if (reply.getPerformative() == ACLMessage.INFORM) {
            // SUCCESS reply from server
            System.out.println("Order confirmed: " + content);
        } else {
            System.out.println("Order refused: " + content);
        }
    }

    private void handlHistory(String historyContent) {
        System.out.println("\n" + getLocalName() + ": Received History");
        String[] items = historyContent.split("\\|\\|");
```

```
        for (int i = 1; i < items.length; i++) { // skip "History" header
            if (items[i].trim().isEmpty()) continue;
            String[] parts = items[i].split(":");
            if (parts.length < 3) continue; // safety check
            String clientName = parts[0];
            String bookName = parts[1];
            int qty = Integer.parseInt(parts[2]);
            System.out.println("OrderItem: " + bookName + " x" + qty + " by " + clientName);
        }
    }
}

```